Paper Review • arXiv 2611.12885 • cs.CV • 2026-11-09

World In-Context Learning for Agentic Robot Manipulation

Robots Learn In-Context: 1 Demo + Sim Rehearsal, 0 Gradients

Heng Fang, Zehui Guo, Yuehan Zhang, Mingyang Du, Dingkang Liang, Xiang Bai
Huazhong Univ. of Sci. & Tech. • ArcheOne AI Code: github.com/ArcheOne-AI/RoboScript

TL;DR

In short: skills in readable context beat weights across distributions

A frozen foundation model starts from **1 demonstration**, **rehearses** the task in a simulator, and writes world feedback into a semantic **Task Memory**; with no gradient updates, it beats every action policy trained on 50 demos across all 50 RoboTwin 2.0 tasks.

1

demo / task

vs. 50 for baselines

0

gradient updates

Weights frozen throughout

83.4%

Avg. success, 50 tasks

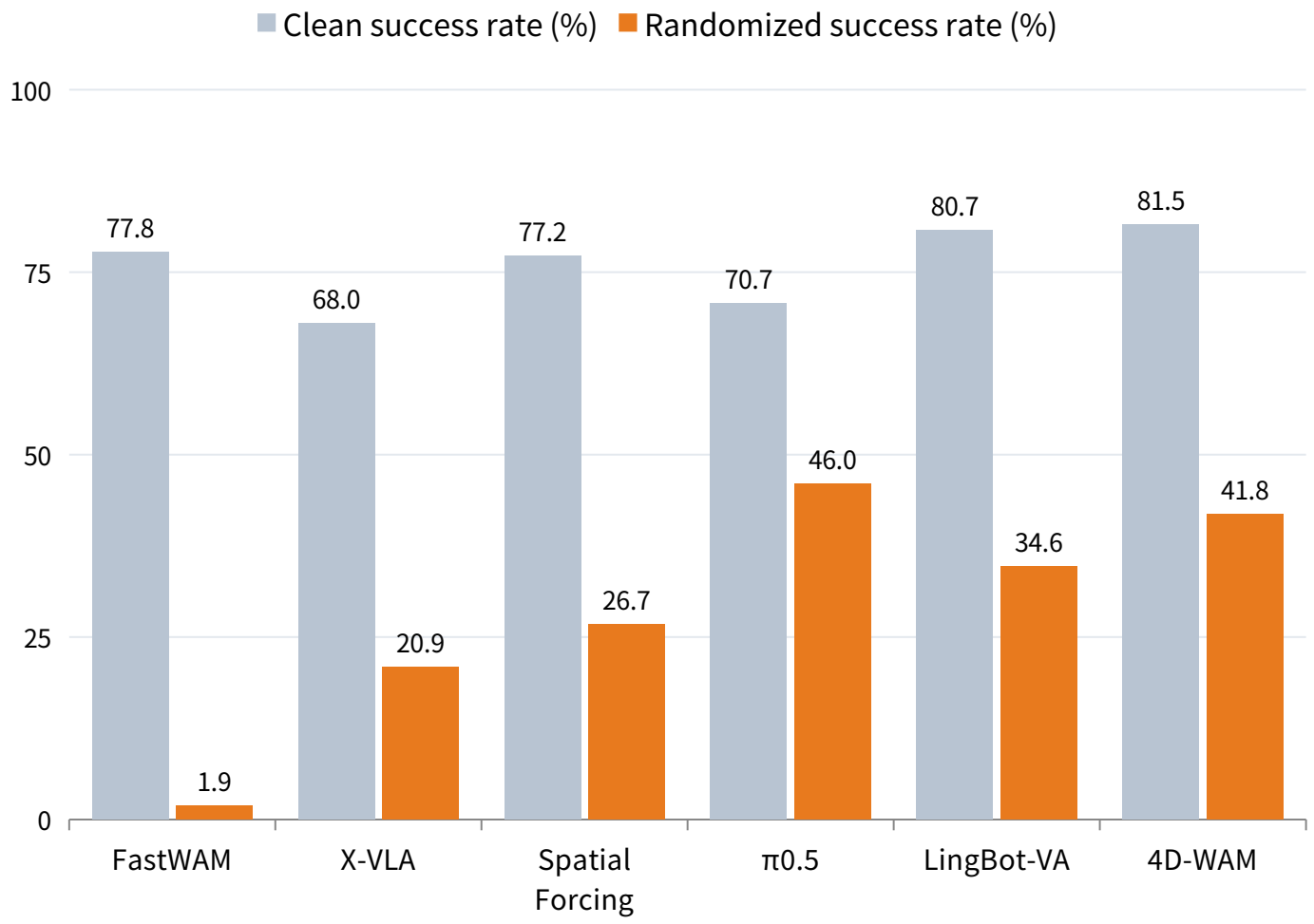
Best trained policy: 61.7%

99.5%

Rand. retention

Trained policies: ⩽65.1%

Problem: policies bind skills to training data and break under shift



FastWAM

77.8% → 1.9%

clean → domain randomization

Weight learning

Skills live in network parameters, coupled to the training distribution; new tasks/distributions require new data and retraining.

Conventional ICL

Demos come from other scenes, with no feedback from the current world; GPT-6 Astra gains nothing from other-layout demos.

Inspiration: coding agents

They learn a new repo by running code and reading terminal output → robots lack exactly this feedback loop.

Source: Table 1 (trained policies use 50 clean demos per task)

KEY INSIGHT

Key insight: context-learned semantics transfer across worlds

Learning in Weights

Gradient training; skill in θ

Bound to training domain

In-Context Learning

Demo placed in context C

Bound to the shown scene

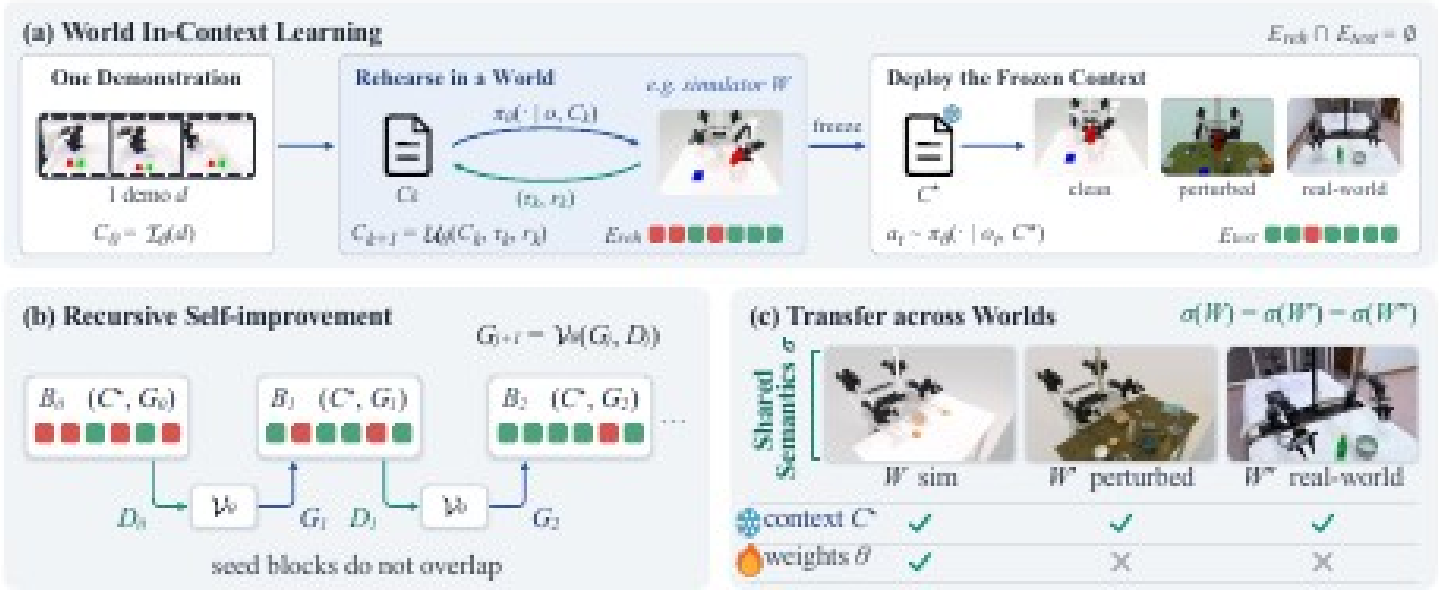
World In-Context Learning

Rehearse in world; skill written to C

Bound to task semantics → transferable

Task Memory records object roles, task stages and lessons, not visual appearance (Fig. 1b: simulator ≈ terminal, Task Memory ≈ AGENTS.md)

WICL: ICL + world feedback; frozen weights, only context changes



① Rehearse & Revise

$C_0 = I_\theta(d)$

$(\tau_k, r_k) \sim W(\pi_\theta(\cdot \mid o, C_k), e_k)$

$C_{k+1} = U_\theta(C_k, \tau_k, r_k)$

$e_k \in E_{reh}$; one frozen model initializes and updates

② Frozen Deployment

$a_t \sim \pi_\theta(\cdot \mid o_t, C^*)$

$e \in E_{test}, \; E_{reh} \cap E_{test} = \varnothing$

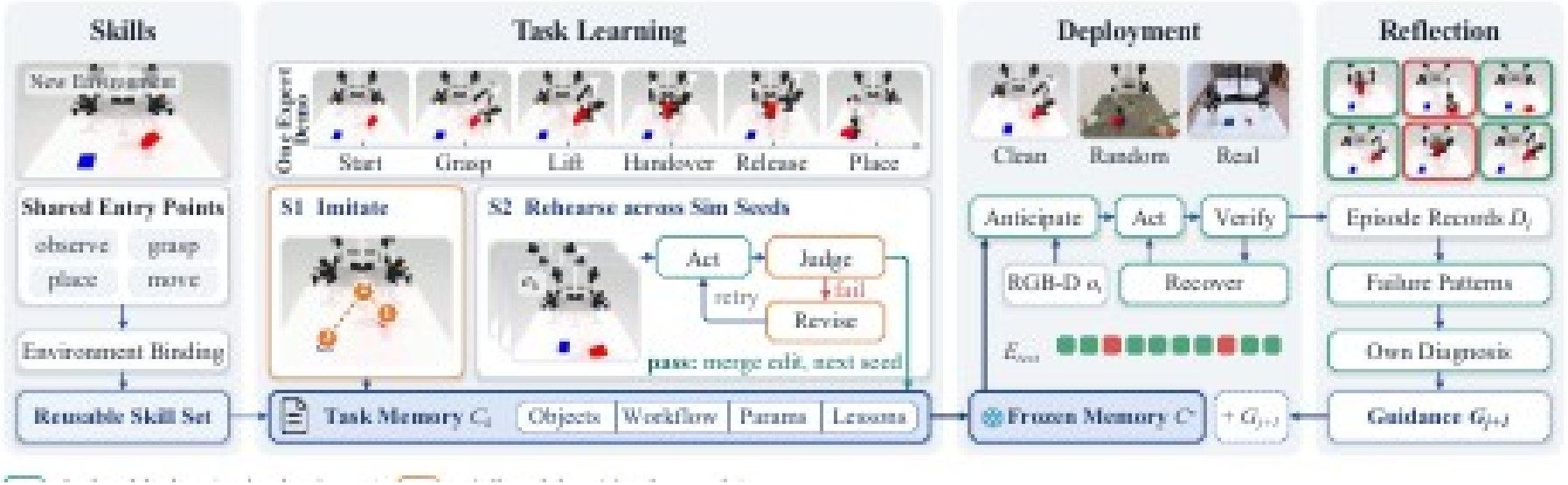
Test instances never rehearsed → tests generalization, not memorization

③ Recursive Self-Improvement

$G_{j+1} = V_\theta(G_j, D_j)$

Reflect on disjoint eval blocks; logs hold only deploy-visible info and outcomes

RoboScript: Skills → 1-Demo Learning → Frozen Deploy → Reflect



1 Skills

Shared entry points + env. bindings; a coding agent writes skills for demo segments it cannot explain, frozen after passing gates

2 Task Learning

S1 Imitate: read the demo, write memory, practice until stable

S2 Rehearse: on new seeds only the first attempt counts; revisions merge only if they make that instance succeed

3 Deployment

Scene pre-check (role mapping, arm choice, feasibility) + reset-free failure recovery + completion check vs. demo end state

4 Reflection

Read deployment logs (visible info + outcomes only), self-diagnose failure modes, write Guidance for the next block

Task Memory: linter-gated Markdown, semantics + relative geometry only

place_object_basket • frozen Task Memory (excerpt, GPT-6.1-Sol)

INSTRUCTION	Grab the bright sleek toycar, drop it in the basket ...
OBJECTS	toycar: the object to drop into the container ... basket: receives the object and is then moved ...
WORKFLOW	1 prepare → 2 grasp → 3 handover → 4 lift → 5 drop → 6 retreat → 7 grasp_basket → 8 move_basket
PARAMETERS	toycar_drop = place_spec(target=basket top_center, frame=target, ...) + 19 more specs
LESSONS	L2 ... keep the arm selected for the object through retreat ...
NOTES	The demonstration ends with the loaded basket still grasped ...

Relative specs, not poses

GraspSpec / PlaceSpec reference perceived object parts, e.g. "approach the handle from the left, grasp its midpoint"; solved on current geometry at run time and mirrorable to the other hand.

Memory linter

Rejects coordinates, absolute positions, distances to the robot, simulator names, eval terms, instance IDs, fixed arm assignments.

Rehearsal writes memory

Here, stage 3 (hand-over) and two lessons were added during rehearsal.

Action receipts

Each action returns commanded pose, actual pose, gripper opening and holding status—from proprioception and planner, also available on real robots.

Data isolation: act on real-robot inputs; privileged info only supervises

All model calls go through a unified Model Gateway, so swapping foundation models is a config change; the Agent plane uses only deployable interfaces (D), and only the Control plane accesses privileged data (P); methods, benchmarks and embodiments are swappable plugins.

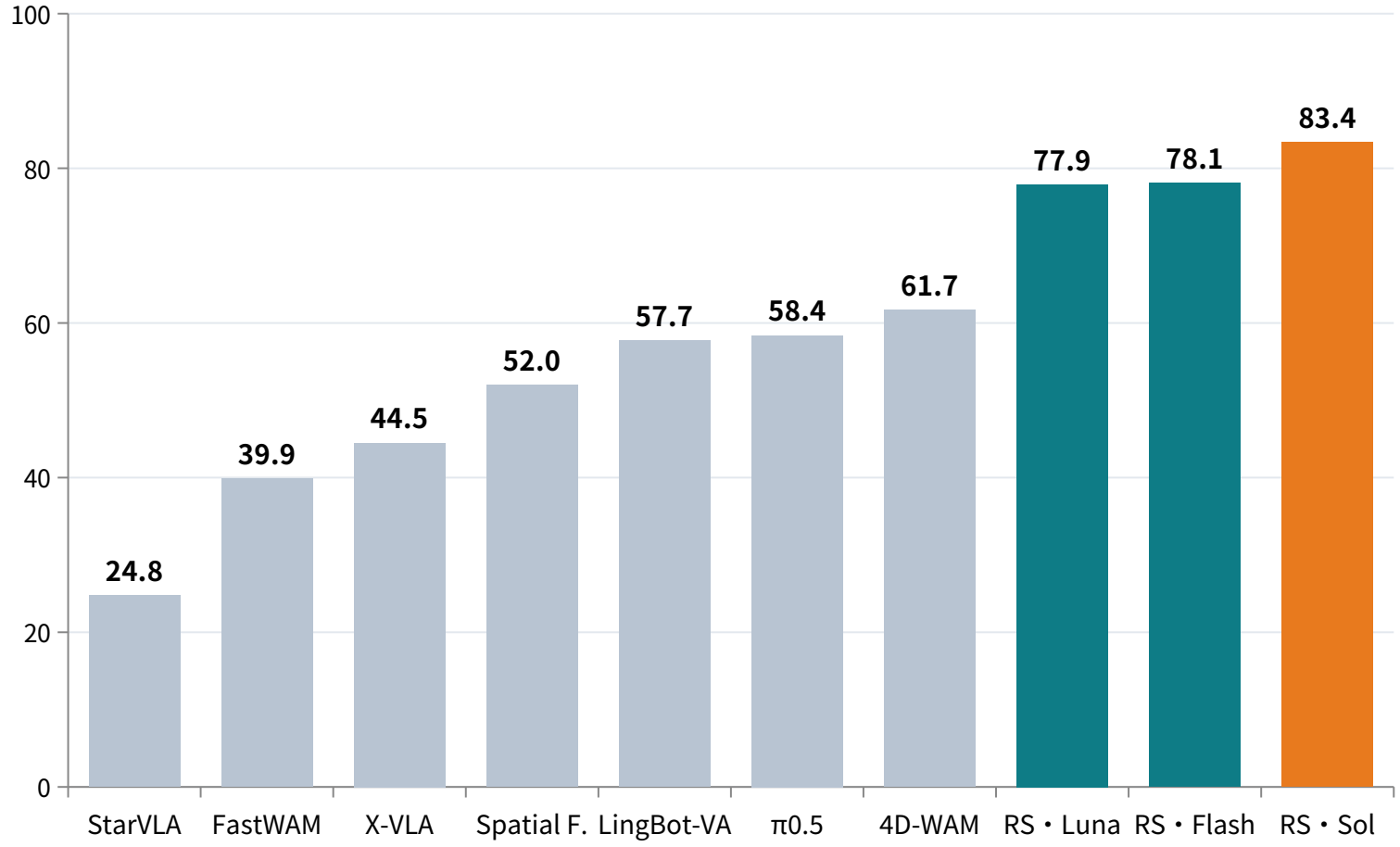
Stage	Role	Data	Edits mem.
S1	Demo reader	P	Yes
S1	Executor	P (demo as hint)	No
S1	Reviewer	P	Yes
S2	Executor	D	No
S2	Reviewer	D + GT diagnosis	Cached copy
Deploy	Executor	D	No

Table: data available to each role (from Table 5)

Isolation test: removing all privileged supervision (demo GT states + failure GT diagnosis) yields a 10-task mean of **86.5%** vs. **86.0%** for the full method—performance does not come from privileged info.

Result: 1 demo, 0 gradients, 83.4% avg., +21.7 pts over best baseline

Clean & Rand avg. success (%) Gray: trained action policies (50 demos) Teal/Orange: RoboScript (1 demo)



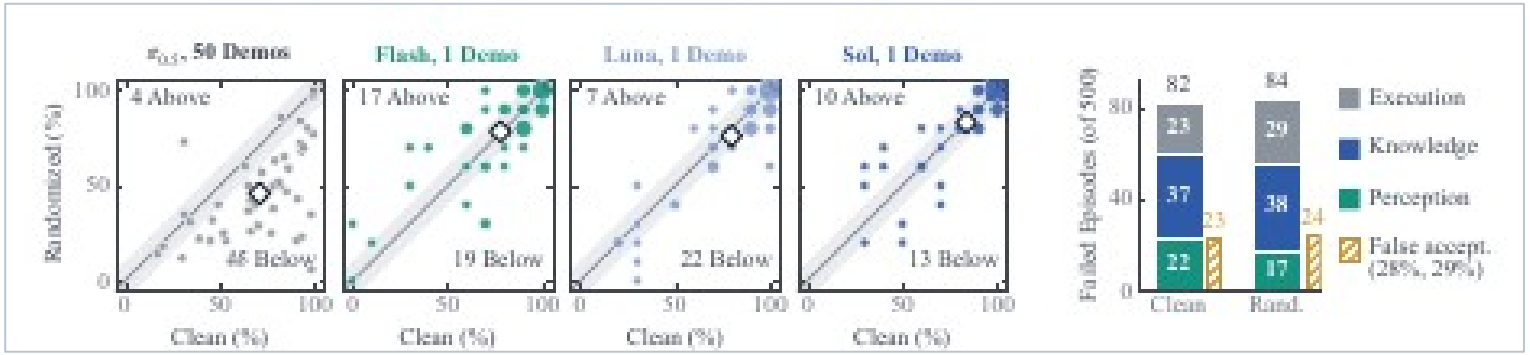
Model	Clean	Rand.	Retention
GPT-6.1-Sol	**83.6**	**83.2**	**99.5%**
GPT-5.6-Luna	79.8	76.0	95.2%
DeepSeek-V4.1-Flash	77.8	78.4	100.8%
4D-WAM (best baseline)	81.5	41.8	51.3%

Key points

- On par with 4D-WAM on clean (83.6 vs 81.5); the gap comes mainly from randomization (83.2 vs 41.8)
- Sol 95% Wilson CI: 81.0–85.6
- All three models, with the same code/prompts/protocol, beat every trained policy

Source: Table 1 (RoboScript: 10 episodes per task per setting; baselines: 100)

Clean-only rehearsal, slight drop when randomized: failures are physical



(a) Per-task clean vs. randomized: π0.5 has 45 tasks below the diagonal; RoboScript's three models scatter on both sides. (b) Sol failures by root cause.

Interpretation: random table heights make actions harder (more execution failures) but do not affect "what it knows"; false acceptance stays at 28% / 29%.

95.2–100.8%

Randomization retention of the three models; trained policies: ⩽65.1%

37 → 38

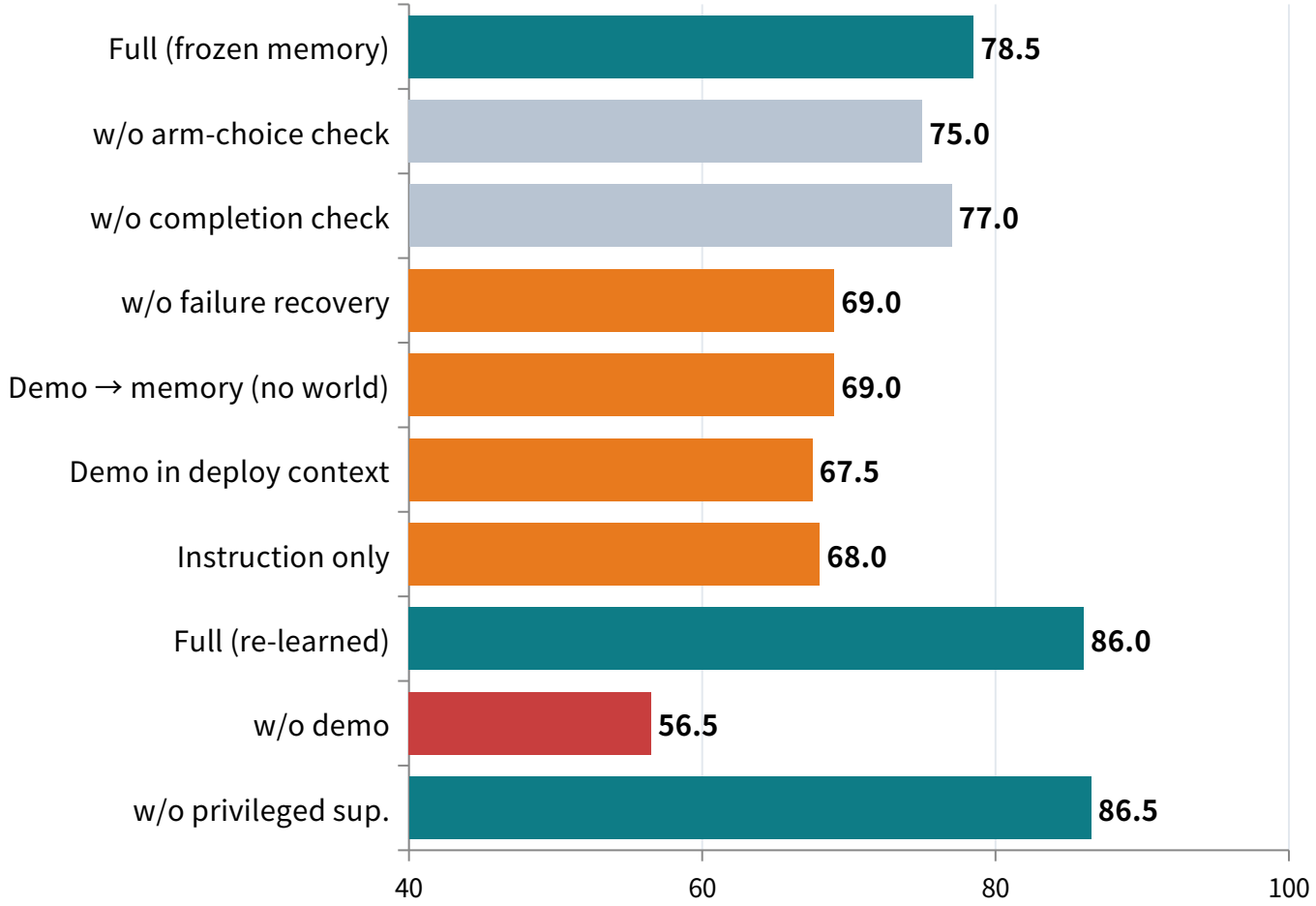
Sol knowledge failures (clean → rand) nearly unchanged; perception 22→17, execution 23→29

78%

Share of episodes with unseen object instances in variable-object tasks; no significant success difference (CMH p=0.97)

Ablation: both demo and world are needed—demos alone barely help

Avg. success (%, mean of clean and rand); each variant paired against its group's "Full"



World is essential

Same demo: no world interaction 69.0% vs. with rehearsal 78.5% (p=0.003); three "no-world" variants differ by ⩽1.5 pts.

Demo is the anchor

Rehearsing from scratch without the demo: 86.0% → 56.5%, the largest drop; some tasks fail to converge.

Rehearsal fixes details

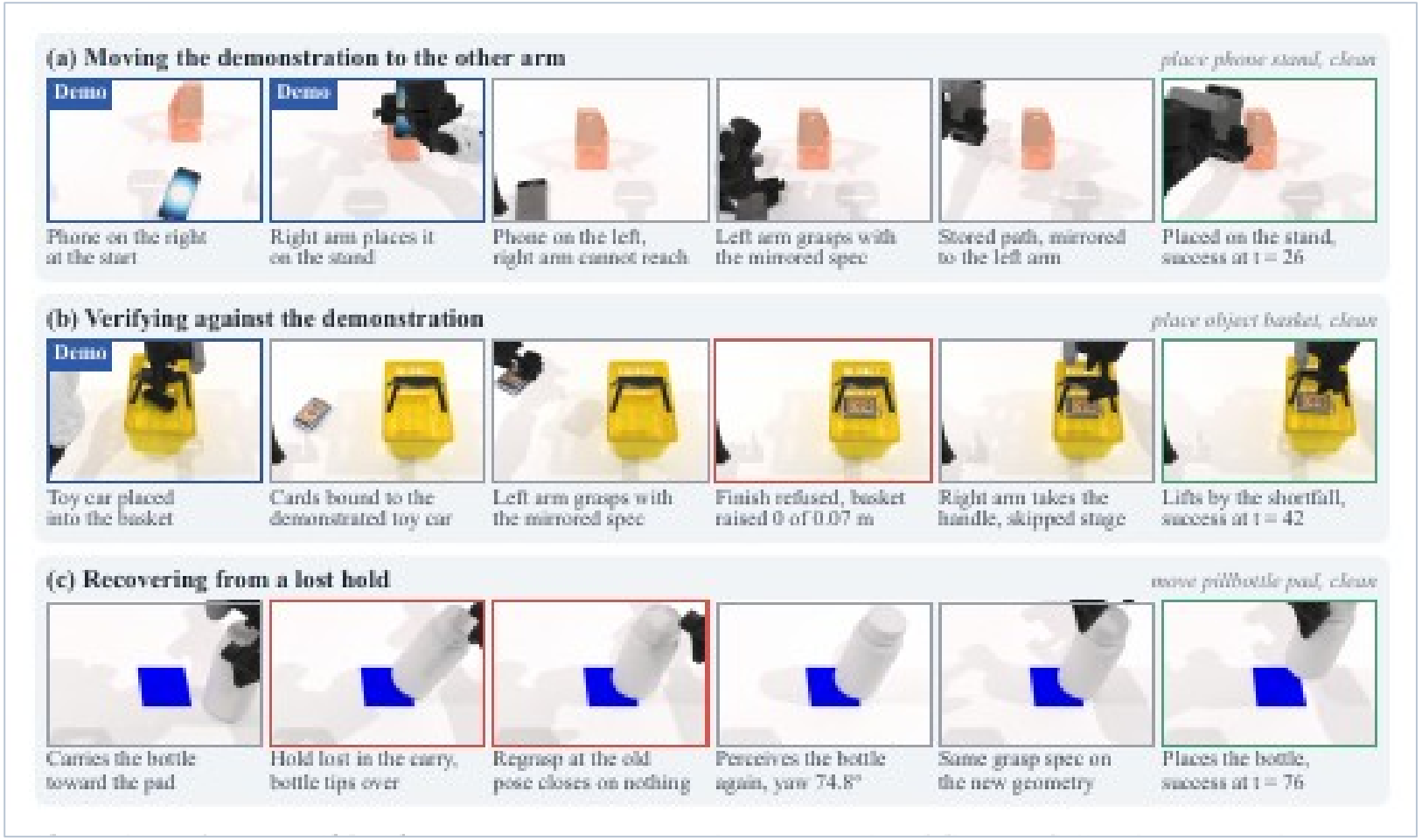
On 11 hard tasks, rehearsal adds +8.4 pts, and +13.3 pts under randomization (p=0.034).

Deployment components

Removing failure recovery hurts most (78.5 → 69.0); removing privileged supervision has no effect (86.5).

Source: Table 2 (GPT-6.1-Sol, 10 tasks × 10 episodes × 2 settings, paired comparison)

Deployment: mirrored arm swap, early-stop refusal, regrasp after drop



(a) Arm-choice check

When the object is on the other side, out of the demo arm's reach, the other hand grasps with mirrored specs; ~half of successful grasps switched arms, with similar success (93% / 95%).

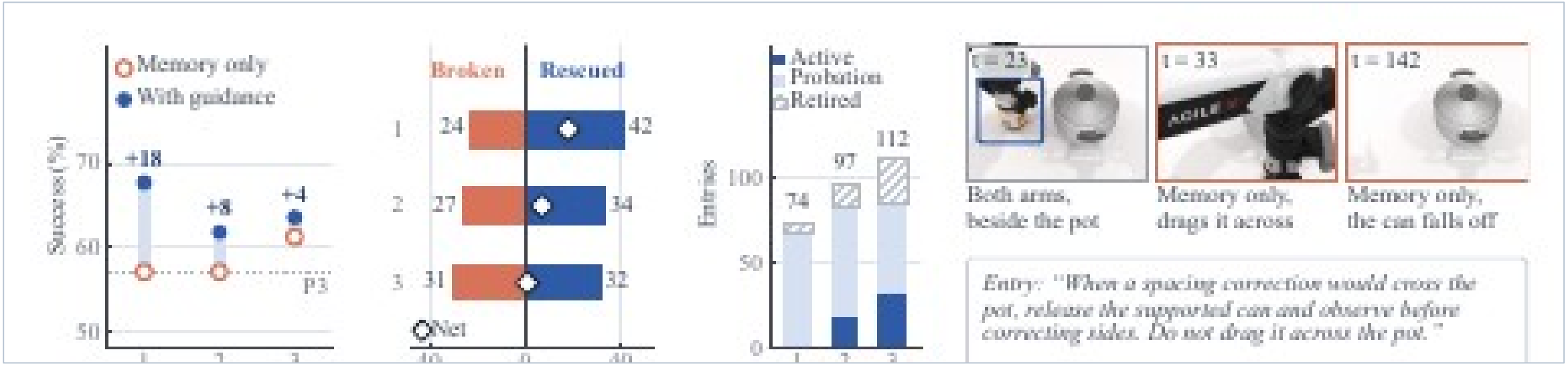
(b) Completion check

Refuses to finish when the end state mismatches the demo (basket not lifted) and returns to the skipped stage.

(c) Failure recovery

After a drop during transport, no env. reset: re-perceives the bottle pose and regrasps on the new geometry with the same grasp spec, then completes.

Frozen-weight self-improvement: reflective Guidance nets gains each round



+18 / +8 / +4

Net gain over 3 rounds on 17 weak tasks
(rescued − broken episodes)

+18 / +7 / +1

Net gain on all 50 tasks
Gains decay quickly over rounds

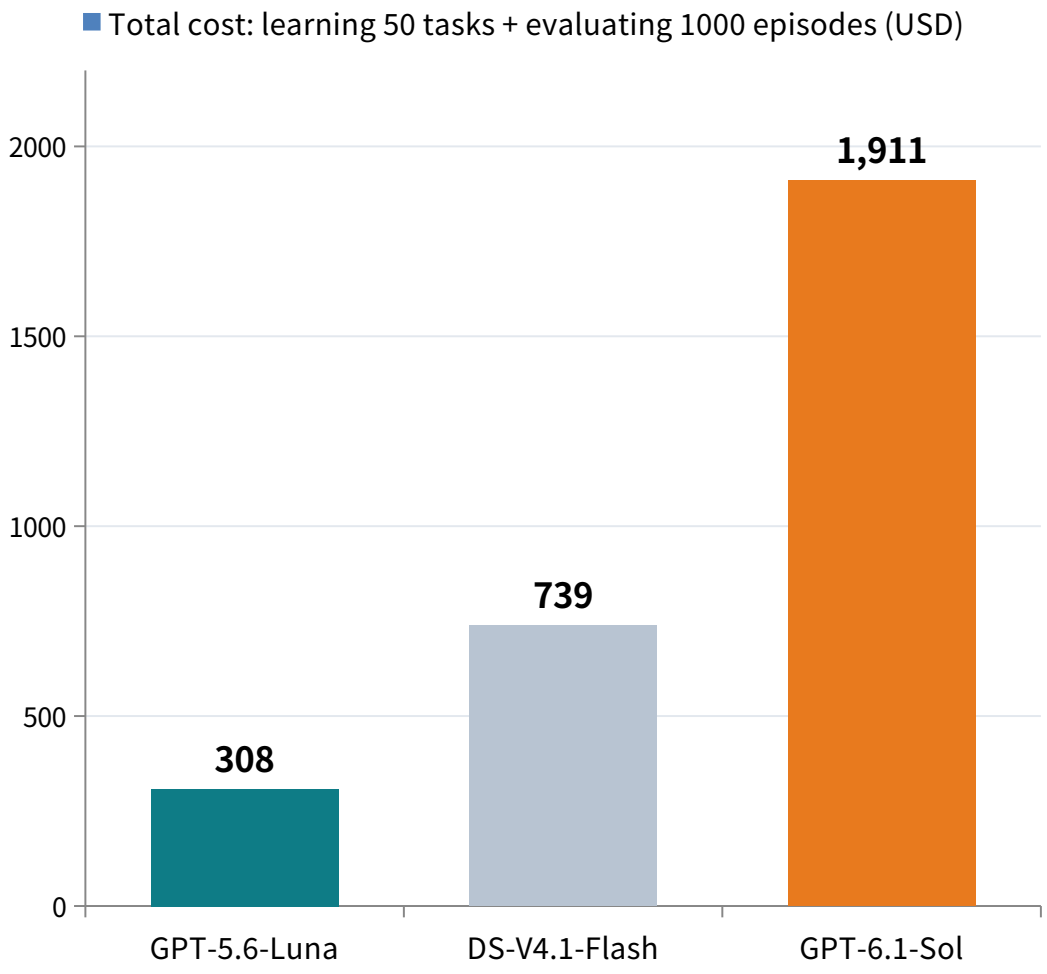
74 → 97 → 112

Growth in Guidance entries
(active / probation / retired)

4/10 → 10/10

handover_block: two guidance items
Effect on identical seeds (Fig. 16)

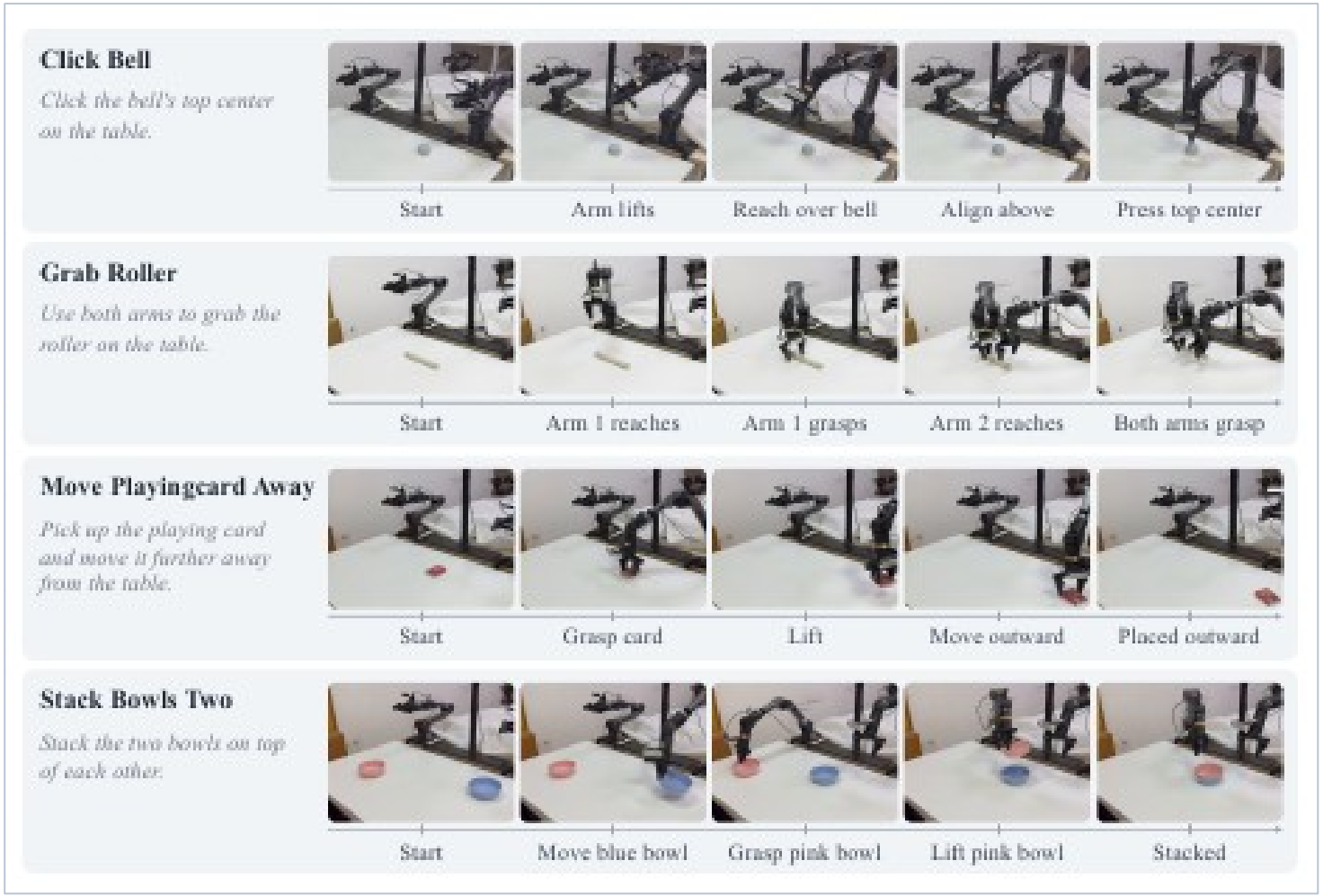
Cost: ~55 model calls, millions of tokens and several minutes per episode

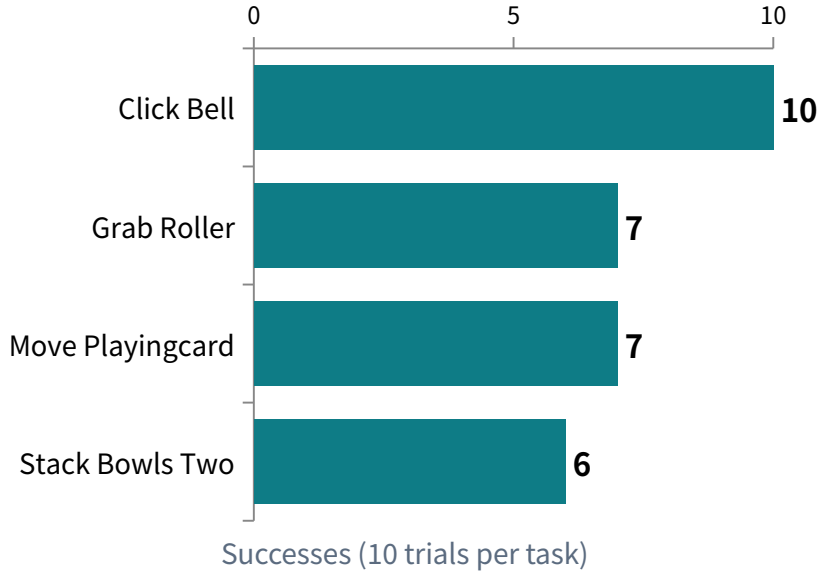


Per episode (eval)	Flash	Luna	Sol
Avg. success %	78.1	77.9	83.4
Model calls	53.1	62.2	54.7
Tokens (M)	3.89	2.61	2.33
Sim steps	183	190	149
Median time (min)	3.6	10.4	6.4
Output tokens / call	4,750	455	245

- Eval. dominates total cost; ~80% of time waits on the base model
- Flash is cheap per token but outputs ~19× Sol per call; total exceeds Luna
- Harder tasks use more tokens; frozen memory reused, no relearning

Real robot: sim memory → heterogeneous dual-arm PiPER via a light adapter





How

Adapter only alters coord. frames, kinematics, gripper params; workflow, object roles, relative specs kept; no new demos/rehearsal.

Note

Planned offline, no online replanning; main failures: gripper misalignment, real depth error.

ASSESSMENT

Verdict: clear paradigm, solid engineering, but uneven comparisons

Strengths

- Clear paradigm: unifies IL / RL / ICL / WICL by "learning signal from demo or world × knowledge in weights or context"
- Anti-memorization: rehearsal and test instances are disjoint; only first attempts count
- Data isolation guaranteed by construction; no change without privileged supervision (86.5 vs 86.0)
- ~100% randomization retention; failure attribution shows knowledge failures don't shift with distribution
- Memory is readable text: auditable, editable, and self-improvement acts directly on it

Caveats

- Unequal comparison: relies on depth-based geometric perception + motion planner + 3-tier API, and env. skills were written by a coding agent using demos from all 50 tasks; "1 vs 50 demos" hides this prior
- Robustness sources not disentangled: depth perception itself is insensitive to texture and lighting
- Eval scale: 10 episodes per task per setting (baselines: 100); API service quality shifts success by several points
- Slow and costly: minutes and millions of tokens per episode
- Only 4 simple real-robot tasks, offline planning; deformable objects and tight bimanual coordination remain hard

SUMMARY

Takeaways: with the world in the loop, skills need not live only in weights

1	**Paradigm**	WICL = ICL + world feedback: freeze weights, rehearse in a resettable, faithful world, and write semantic knowledge into context.
2	**Evidence**	83.4% avg. on all 50 RoboTwin 2.0 tasks (1 demo, zero gradients), ~99.5% randomization retention; all three foundation models beat trained policies.
3	**Mechanism**	The demo gives a starting point, the world gives corrections; the linter and data isolation let memory move to real robots (PiPER, 4 tasks, 6–10/10).
4	**Next steps**	Authors propose rehearsal in learned world models, automatic skill expansion, multi-agent collaboration, and foundation models trained for embodied tool use, making WICL faster and cheaper.